NATIONAL LAND ACQUISITION & MANAGEMENT SYSTEM (NLAMS) · TELANGANA

Project Development Report

How multiple tools were merged into one complete, working product

SIH 2026

PS 26016

Smart Automation

Project	**NLAMS — Real-Time Land Acquisition & Management System (Telangana prototype)**
Organization / PS	Ministry of Rural Development · Dept. of Land Resources (DoLR)
Team	<Team Name> · Team ID <xxxx> · <College / Institute Name>
Stack summary	Figma · QGIS/GeoJSON · Python · HTML5/CSS3/JS · SVG GIS engine · Node.js API · Git · Postman
Deliverables	frontend/ (SPA) · backend/ (REST API) · dist/ (single-file build) · docs/ (SIH decks)

1 · TOOLS & TECHNOLOGIES USED

Every tool, its role, and what it contributed to the final build

DESIGN & PLANNING		
Figma	UI/UX wireframes of login, dashboard & GIS screens	→ color tokens & layout → css/styles.css
Notion / Docs	requirement study of RFCTLARR Act 2013	→ 7-stage workflow model → js/01-config.js
GIS DATA & PROCESSING		
QGIS + Python 3	boundaries of all 33 districts; projection to SVG, simplification, centroids	→ data/telangana.geojson → js/03-map-data.js
FRONTEND		
HTML5 + CSS3	semantic layout, light government theme, responsive grid	→ frontend/index.html + css/styles.css
JavaScript (ES6)	13 ordered modules — state machine, GIS engine, views	→ frontend/src/js/01…13
SVG (in-house engine)	live vector map: route splines, parcel corridors, zoom/pan	→ 10-views-gis.js + 05-engine.js
BACKEND		
Node.js (std-lib http/vm/fs)	REST API over the shared domain core — zero npm installs	→ backend/src/server.js + routes/ + controllers/
PostgreSQL + PostGIS	production data model (parcels with geometry, SLA payments, audit)	→ backend/src/models/schema.sql
QUALITY & DELIVERY		
Git + GitHub · DevTools	version control & team collaboration; layout/console debugging	→ commit history · .gitignore
Postman/cURL + smoke.js	API contract tests (11 endpoints) + auto regression of every page/tab	→ scripts/smoke.js → SMOKE PASS

2 · HOW THE TOOLS MERGED INTO ONE PROJECT

INPUT TOOLS designs · geo-data · code	**Figma** login & dashboard mockups	**QGIS / GeoJSON** 33 district boundaries	**VS Code** 13 JS modules + API server
TRANSFORM TOOLS python · git	**build_map.py** GeoJSON → SVG paths + centroids	**domain core** one business logic shared by UI & API	**git commits** merge every contribution safely
BUILD & MERGE one command	**scripts/build.py** bundles CSS + 13 modules + assets	**node backend** serves SPA + /api on one port	**smoke.js** renders every page automatically
COMPLETE PROJECT deliverables	**frontend/ + backend/** full-stack source tree	**dist/index.html** single-file offline build (484 KB)	**docs/** SIH PDF/PPT + this report

Key idea: no tool output lives alone — every artefact is merged by the pipeline into one tested, single-command product (npm start · npm run build).

3 · SYSTEM ARCHITECTURE

One shared business core — the UI renders it, the API serves it, nothing drifts

BROWSER — frontend/ (vanilla JS SPA, zero frameworks)

Views 07–12

login · dashboard · GIS map · projects · acquisition · compensation

GIS engine (05)

route splines · parcel corridors · aggregates

State (06)

single S object · render loop · sessionStorage

▼ fetch /api/* · JSON

SERVER — backend/ (Node std-lib, zero npm deps)

controllers/

request handling & validation

services/domain.js

loads the SAME js modules in a VM — shared logic

▼

DATA — today & production

Prototype: 04-data.js in-memory store (14 projects · 5 alerts · deterministic parcels) + data/telangana.geojson.

Production: PostgreSQL + PostGIS (models/schema.sql) integrating Dharani · Bhuvan · CFMS · e-Gazette.

- Why this wins — the domain logic is written once (plain JS modules) and loaded by BOTH the browser and the Node API via a VM context — the exact guarantee juries ask about ("how do UI and API stay in sync?").
- Offline-first — npm start serves everything on :8080; dist/index.html alone works from a pendrive with no server at all.

4 · PROJECT STRUCTURE (AS BUILT)

tree output of the final repository — every file has one job

```
nlams/
|-- package.json                npm start | test | build | map | deck
|-- README.md                   run guide | API table | architecture notes
|-- frontend/
|   |-- index.html              SPA entry (loads the src tree)
|   +-- src/
|       |-- css/styles.css      theme | components | GIS overlays
|       |-- assets/             emblem.png - bg.jpg
|       +-- js/
|           |-- 01-config.js    stages - colours (RFCTLARR model)
|           |-- 02-utils.js     helpers - icons - toasts - modals
|           |-- 03-map-data.js  Telangana geometry (GENERATED)
|           |-- 04-data.js      demo dataset (14 projects - alerts)
|           |-- 05-engine.js    GIS ENGINE: routes - corridors - KPIs
|           |-- 06-state.js     app state - session persistence
|           |-- 07..08-views    login - shell
|           |-- 09..12-views    dashboard - GIS - projects - modules
|           +-- 13-app.js       render loop - events - boot
|-- backend/
|   +-- src/  server.js - routes/ - controllers/ - services/domain.js
|             models/schema.sql (production PostGIS schema)
|-- data/telangana.geojson      raw Survey-of-India boundaries
|-- scripts/                    build_map.py - build.py - smoke.js - decks
|-- dist/index.html             single-file offline build (484 KB)
+-- docs/                       SIH submission PDF/PPTX - pitch deck - report
```

5 · WHAT WAS BUILT — MODULES & FEATURES

every screen is functional and stateful; actions propagate live across pages

LOGIN & ROLES

Emblem-branded light portal on photo background; 4 government roles + demo access; session persists across refresh.

STATE DASHBOARD

6 KPI cards, live Telangana status map (click district → GIS), district progress bars, sector chart, alert feed.

GIS LAND MAP ■

All 33 real district boundaries · 3 colour modes · zoom/pan · district drill-down · project ALIGNMENT view: route spline, green = acquired, parcels coloured by workflow stage, stage-click filtering.

ACQUISITION WORKFLOW

Proposal form → Scrutiny review (approve/reject) → Sec-11 notification publishing → parcel-by-parcel GPS verification → RFCTLARR award calculator (multiplier + 12% + solatium) → possession notices.

COMPENSATION

CFMS-DBT ledger per family; one-click release updates state totals; 90-day SLA warnings.

PROJECT DETAILS

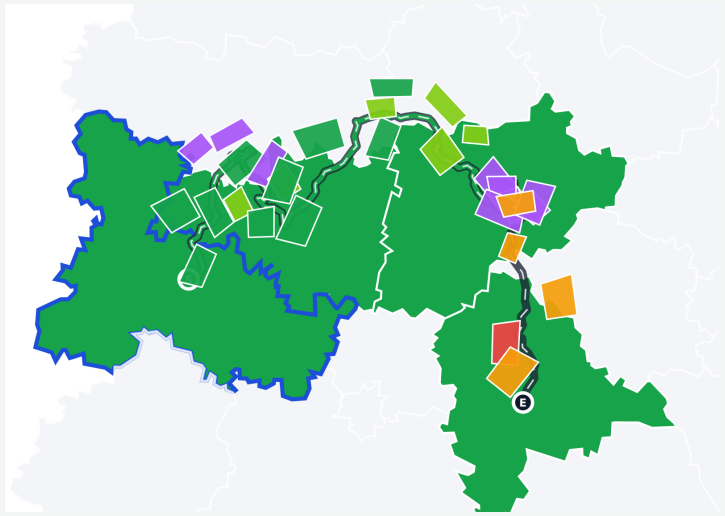
Single source of truth per project: overview, corridor parcel map, stage timeline, payments, alerts — 5 tabs.

ALERTS & RISK

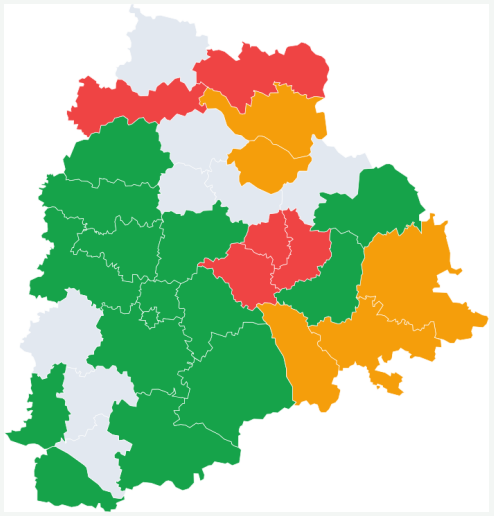
Risk-scored alerts (0–100) with recommended actions; dashboard cross-links into each case.

6 · THE RESULT — SCREENSHOTS FROM THE BUILD

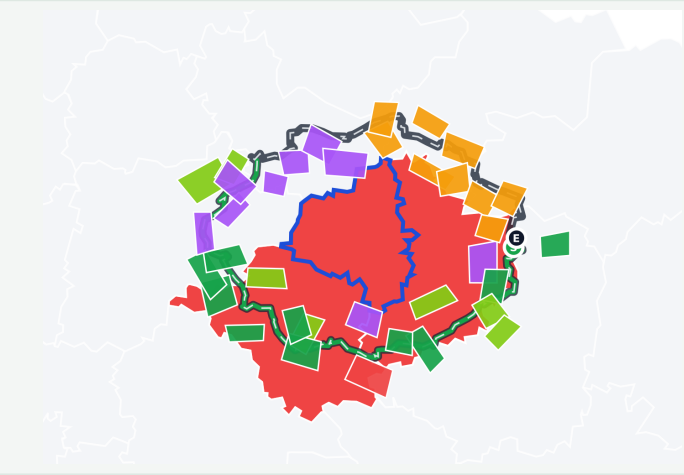
not mockups: rendered by the actual merged application



Route-alignment GIS — Regional Ring Road corridor with parcels in workflow colours



Dashboard map — live status of all 33 districts



Warangal ORR — closed-ring alignment, acquisition visible around the ring

Verified by automation

- ✓ smoke.js → renders 14 projects × 5 tabs + 6 workflow pages + GIS views → SMOKE PASS
- ✓ API → 11 endpoints tested via cURL/Postman (health · kpi · districts · search · alignment · writes)
- ✓ Map engine → geometry asserted inside viewBox; parcels deterministic (same seed = same map)
- ✓ Session → refresh keeps user on the same page (storage-safe fallback)

7 · FUTURE SCOPE & REFERENCES

prototype → production path is fully mapped

Phase 1 — field pilot

Telugu UI · mobile app with offline GPS parcel capture for field officers

Phase 2 — integrations

Dharani land records · CFMS payment rails · e-Gazette publishing · Bhuvan cadastral layers

Phase 3 — intelligence

ML risk scoring from delay patterns · SMS/WhatsApp status to affected families

Phase 4 — national grid

state-agnostic engine (boundaries as config) → DoLR multi-state deployment

References

- RFCTLARR Act 2013 — dolr.gov.in · indiacode.nic.in
- DILRMP — dolr.gov.in/programmes/dilrmp
- Dharani — dharani.telangana.gov.in
- CFMS — cfms.telangana.gov.in
- Bhuvan (ISRO) — bhuvan.nrsc.gov.in
- PM Gati Shakti — gatishakti.gov.in
- SIH 2026 PS 26016 (official statement)

Closing note: every tool in section 1 left an artefact; every artefact is merged by the pipeline in section 2; the result is a working product a jury can click through in under 3 minutes — or run offline from a single file.